

Project Blueprint: Global Logistics & Supply Chain Optimizer

A Three-Tier, Real-Time, Role-Aware Logistics Intelligence Platform

1. Project Objectives

1.1 Real-Time Vehicle Tracking

- Visualize simulated truck movement on a map.
- Consume periodic GPS updates via FastAPI, displayed through Flask dashboards.

1.2 Inventory Management Across Warehouses

- Track stock levels and validate availability before shipment creation.
- Maintain accurate quantities across distributed storage centers.

1.3 Order Lifecycle Automation

- Manage shipments through distinct states: \ **Pending** → **Assigned** → **In Transit** → **Delivered**

1.4 Role-Based Access Control

- **Admin Users:** Full visibility of all trucks, inventory editing, and shipment creation.
- **Drivers:** Limited views of their assigned vehicle and status updates only.

2. System Architecture – Three-Tier Stack

Layer Overview

Layer	Component	Responsibility
Presentation	Flask	UI rendering, session & cookie handling, dashboards, visualizations
Logic / API	FastAPI	Business logic, data validation, telemetry ingestion, API endpoints
Data	MS SQL Server	Relational storage, temporal tables, data integrity

Architecture Flow

1. **User interacts with Flask UI** →
2. **Flask calls FastAPI using HTTPX** →
3. **FastAPI reads/writes MS SQL via SQLAlchemy ORM** →
4. **Simulated trucks push telemetry to FastAPI** →
5. **Flask visualizes results back to users**

3. Detailed Feature Descriptions

F1: Secure Authentication System

Description: \ A centralized login service controlling access with clear role separation.

Logic Flow:

1. User submits credentials to FastAPI.
2. FastAPI validates and returns a **JWT token**.
3. Flask stores token in a secure cookie.
4. Every subsequent call from Flask → FastAPI includes the token in headers.
5. FastAPI enforces RBAC (Role-Based Access Control).

Roles:

- **Admin:**
 - Manage trucks
 - Modify inventory
 - Create and assign shipments
- **Driver:**
 - View assigned vehicle
 - Update delivery status

F2: Warehouse & Inventory Control

Description: \ Digital twin model for warehouse operations.

Logic:

- Each product is linked to a WarehouseID.
- When a shipment is created, FastAPI checks if inventory is sufficient.

- If stock is adequate → shipment proceeds.
- If not → creation is rejected with an error.

F3: Live Telemetry & Simulation Engine

Description: \ A Python script simulating realistic truck movement using GPS coordinates.

Logic:

- The script calculates the distance between two geospatial points.
- Every few seconds POSTs updated coordinates to a FastAPI telemetry endpoint.
- FastAPI updates the TelemetryLogs table and the truck's latest known location.

F4: Smart Routing & Dispatch

Description: \ AI-assisted decision engine that finds the best vehicle for a new shipment.

Logic:

- FastAPI queries database for eligible vehicles.
- Computes distance between vehicle and warehouse.
- Selects the closest “Available” vehicle.
- Assigns the VehicleID to the shipment automatically.

F5: Analytical Reporting Dashboard

Description: \ A manager-focused performance overview delivered via Flask.

Dashboard Includes:

- Total completed shipments
- Simulated fuel efficiency metrics
- Inventory heat map showing warehouse activity levels
- Truck utilization statistics

Data Flow: \ Flask → HTTPX → FastAPI → SQL → FastAPI → Flask

4. Data Entity Map (Database Schema Skeleton)

Core Tables

1. Users

- a. UserID
- b. Username
- c. PasswordHash
- d. Role

2. Warehouses

- a. WarehouseID
- b. Name
- c. Latitude
- d. Longitude
- e. Capacity

3. Products

- a. ProductID
- b. Name
- c. SKU
- d. Weight

4. Inventory

- a. InventoryID
- b. WarehouseID (FK)
- c. ProductID (FK)
- d. Quantity

5. Vehicles

- a. VehicleID
- b. Type
- c. Status (Idle / Moving / Repair)
- d. CurrentLatitude
- e. CurrentLongitude

6. Shipments

- a. ShipmentID
- b. WarehouseID
- c. VehicleID
- d. UserID
- e. Status
- f. Timestamps (Created, Assigned, Delivered)

7. TelemetryLogs

- a. LogID
- b. VehicleID
- c. Latitude
- d. Longitude
- e. Timestamp

5. Technical Constraints & Tools

Database

- MS SQL Server (Express or Developer Edition)

ORM

- SQLAlchemy (Core + ORM models)

Frontend

- Flask + Jinja2
- Bootstrap 5 for clean, responsive dashboards

Communication Layer

- HTTPX used by Flask to communicate with FastAPI securely

Telemetry Simulation

- Python script using geodesic distance formulas & timed loops

Final Deliverable Summary

This documentation now includes:\

- ✓ Cleanly structured sections
- ✓ Clear architectural logic
- ✓ Professional formatting
- ✓ Easy-to-follow technical flow
- ✓ Developer-ready database entity map

If you'd like, I can also generate:\

-  A downloadable **PDF documentation**
-  An **architecture diagram** (PNG/SVG)
-  **SQL CREATE TABLE scripts**
-  **FastAPI + Flask starter template code**

6. Life of a Shipment (End-to-End User Flow)

This section describes how data moves across Flask → FastAPI → MS SQL → Simulation Script, showing the full lifecycle of a shipment.

6.1 Shipment Flow Overview

Stage	Technology	Description
1. Creation	Flask	Admin fills out “New Shipment” form (Item, Destination Warehouse).
2. Validation	FastAPI	Flask sends API request → FastAPI checks MS SQL for stock availability.
3. Assignment	FastAPI	Finds the nearest “Available” truck using distance logic.
4. Simulation Trigger	Simulator Script	“Ghost Truck” picks up ShipmentID and begins sending GPS coordinates.
5. Persistence	MS SQL	Each GPS update is stored in TelemetryLogs.
6. Real-Time View	Flask	Flask fetches latest coordinates and renders the moving truck on a map.
7. Completion	FastAPI + MS SQL	Simulation marks delivery → inventory is updated (Source –, Destination +).

6.2 Detailed Step-By-Step Flow

1. Creation (Flask)

Admin logs into Flask dashboard.

Opens New Shipment Form.

Submits: *Product, Source Warehouse, Destination Warehouse.*

2. Validation (FastAPI)

Flask sends POST request → /shipment/create.

FastAPI checks MS SQL:
Does the warehouse have enough quantity?
If no → returns JSON error.
If yes → flow continues.

3. Assignment (FastAPI Business Logic)

FastAPI queries Vehicles table.
Filters trucks where:
Status = "Available"
Calculates distance from each truck → source warehouse.
Selects the nearest truck.
Updates shipment with assigned VehicleID.

4. The Trigger (Simulation Script)

A standalone script (truck_sim.py) periodically queries: GET
/api/shipments/next_assigned
When it detects the new ShipmentID:
Starts moving the virtual vehicle.
Sends GPS coordinates to FastAPI every 5 seconds: POST
/api/telemetry/update

5. Persistence (MS SQL)

Every GPS “ping” is inserted into:

Table	Purpose
TelemetryLogs	Stores all latitude/longitude + timestamp history

FastAPI also updates:

Vehicles.CurrentLatitude
Vehicles.CurrentLongitude

6. Real-Time View (Flask)

Admin refreshes the Shipment Tracking Dashboard.
Flask calls FastAPI: GET /api/telemetry/latest?vehicle=XY123

Renders the updated location on:
Leaflet.js map
OR a custom real-time dashboard

7. Completion (Delivery Event)

When the simulation reaches the destination warehouse coordinates:

Simulation sends: POST /api/shipment/delivered
FastAPI:
Updates Shipments.Status = Delivered
Decreases inventory from Source Warehouse
Increases inventory at Destination Warehouse

Shipment lifecycle is now closed.

7. Professional Monorepo File Structure

The following structure keeps Frontend, Backend, Database, and Simulators cleanly separated inside a single repo:

```
logistics_optimizer/
├── docker-compose.yml    # Runs MS SQL Edge + Redis (M1-compatible)
├── requirements.txt      # All Python dependencies
├── .env                  # Database credentials & secret keys
├──
├── database/            # MS SQL Layer
│   ├── models.py        # SQLAlchemy Table Definitions
│   └── connection.py     # pyodbc connection (ODBC 18)
├──
├── backend/             # FastAPI Layer
│   ├── main.py           # API Routes & Entry Point
│   ├── schemas.py        # Pydantic Validation Models
│   └── services/         # Business Logic (Routing, Inventory, Telemetry)
├──
├── frontend/            # Flask Layer
│   ├── app.py            # Flask Routes & Views
│   └── templates/        # HTML (Dashboard, Login, Maps)
```

```

├── static/          # CSS, JS, Leaflet.js
├── simulator/       # The "Ghost Trucks"
└── truck_sim.py     # Independent GPS Movement Script

```

8. M1 Mac Compatibility Notes

Component	M1 Solution
Database	Use Azure SQL Edge (ARM-compatible) via Docker
Driver	Use ODBC Driver 18 for SQL Server (macOS ARM)
Python Environment	Use venv to isolate dependencies

Why Azure SQL Edge? Standard MS SQL is x86-only. Azure SQL Edge is optimized for ARM-based devices (like M1/M2 Macs).

9. Next Steps: Environment Preparation

Before development, you must prepare your workstation.

Choose your next step:

✓ Option A — Install Docker + MS SQL Edge on M1 (I'll provide complete terminal commands)

✓ Option B — Define the full SQL Schema (All CREATE TABLE scripts in MS SQL format)

Just tell me:

👉 “Give me Option A” or 👉 “Give me Option B” 📦 Project Blueprint: Global Logistics & Supply Chain Optimizer

A Three-Tier, Real-Time, Role-Aware Logistics Intelligence Platform

1. Project Objectives

1.1 Real-Time Vehicle Tracking

- Visualize simulated truck movement on a map.

- Consume periodic GPS updates via FastAPI, displayed through Flask dashboards.

1.2 Inventory Management Across Warehouses

- Track stock levels and validate availability before shipment creation.
- Maintain accurate quantities across distributed storage centers.

1.3 Order Lifecycle Automation

- Manage shipments through distinct states: \ **Pending** → **Assigned** → **In Transit** → **Delivered**

1.4 Role-Based Access Control

- **Admin Users:** Full visibility of all trucks, inventory editing, and shipment creation.
- **Drivers:** Limited views of their assigned vehicle and status updates only.

2. System Architecture – Three-Tier Stack

Layer Overview

Layer	Component	Responsibility
Presentation	Flask	UI rendering, session & cookie handling, dashboards, visualizations
Logic / API	FastAPI	Business logic, data validation, telemetry ingestion, API endpoints
Data	MS SQL Server	Relational storage, temporal tables, data integrity

Architecture Flow

6. **User interacts with Flask UI** →
7. **Flask calls FastAPI using HTTPX** →
8. **FastAPI reads/writes MS SQL via SQLAlchemy ORM** →
9. **Simulated trucks push telemetry to FastAPI** →
10. **Flask visualizes results back to users**

3. Detailed Feature Descriptions

F1: Secure Authentication System

Description: \ A centralized login service controlling access with clear role separation.

Logic Flow:

6. User submits credentials to FastAPI.
7. FastAPI validates and returns a **JWT token**.
8. Flask stores token in a secure cookie.
9. Every subsequent call from Flask → FastAPI includes the token in headers.
10. FastAPI enforces RBAC (Role-Based Access Control).

Roles:

- **Admin:**
 - Manage trucks
 - Modify inventory
 - Create and assign shipments
- **Driver:**
 - View assigned vehicle
 - Update delivery status

F2: Warehouse & Inventory Control

Description: \ Digital twin model for warehouse operations.

Logic:

- Each product is linked to a WarehouseID.
- When a shipment is created, FastAPI checks if inventory is sufficient.
- If stock is adequate → shipment proceeds.
- If not → creation is rejected with an error.

F3: Live Telemetry & Simulation Engine

Description: \ A Python script simulating realistic truck movement using GPS coordinates.

Logic:

- The script calculates the distance between two geospatial points.
- Every few seconds POSTs updated coordinates to a FastAPI telemetry endpoint.
- FastAPI updates the TelemetryLogs table and the truck's latest known location.

F4: Smart Routing & Dispatch

Description: \ AI-assisted decision engine that finds the best vehicle for a new shipment.

Logic:

- FastAPI queries database for eligible vehicles.
- Computes distance between vehicle and warehouse.
- Selects the closest “Available” vehicle.
- Assigns the VehicleID to the shipment automatically.

F5: Analytical Reporting Dashboard

Description: \ A manager-focused performance overview delivered via Flask.

Dashboard Includes:

- Total completed shipments
- Simulated fuel efficiency metrics
- Inventory heat map showing warehouse activity levels
- Truck utilization statistics

Data Flow: \ Flask → HTTPX → FastAPI → SQL → FastAPI → Flask

4. Data Entity Map (Database Schema Skeleton)

Core Tables

8. Users

- UserID
- Username
- PasswordHash
- Role

9. Warehouses

- WarehouseID
- Name

- c. Latitude
- d. Longitude
- e. Capacity

10. Products

- a. ProductID
- b. Name
- c. SKU
- d. Weight

11. Inventory

- a. InventoryID
- b. WarehouseID (FK)
- c. ProductID (FK)
- d. Quantity

12. Vehicles

- a. VehicleID
- b. Type
- c. Status (Idle / Moving / Repair)
- d. CurrentLatitude
- e. CurrentLongitude

13. Shipments

- a. ShipmentID
- b. WarehouseID
- c. VehicleID
- d. UserID
- e. Status
- f. Timestamps (Created, Assigned, Delivered)

14. TelemetryLogs

- a. LogID
- b. VehicleID
- c. Latitude
- d. Longitude
- e. Timestamp

5. Technical Constraints & Tools

Database

- MS SQL Server (Express or Developer Edition)

ORM

- SQLAlchemy (Core + ORM models)

Frontend

- Flask + Jinja2
- Bootstrap 5 for clean, responsive dashboards

Communication Layer

- HTTPX used by Flask to communicate with FastAPI securely

Telemetry Simulation

- Python script using geodesic distance formulas & timed loops

Final Deliverable Summary

This documentation now includes:\

- ✓ Cleanly structured sections
- ✓ Clear architectural logic
- ✓ Professional formatting
- ✓ Easy-to-follow technical flow
- ✓ Developer-ready database entity map

If you'd like, I can also generate:\

-  A downloadable **PDF documentation**
-  An **architecture diagram** (PNG/SVG)
-  **SQL CREATE TABLE scripts**
-  **FastAPI + Flask starter template code**